

Class 12: RNASeq with DESeq2

Ami Pant (A15991358)

Table of contents

Background	1
Data Import	1
Toy analysis example	3
DESeq Analysis	9
Volcano Plot	11
Save our results	12
Add annotation data	12
Save my annotated results	16
Pathway Analysis	16

Background

Today we will analyze some RNASeq data from Himes et al. on the effects of a common steroid (dexamethasone also called “dex”) on airway smooth muscle cells (ASMs)

For this analysis we need two main inputs

- **countData**: a table of **counts** per gene (in rows) across experiments (in columns)
- **colData**: **metadata** about the design of the experiments. The rows match the columns in **countData**

Data Import

```
counts <- read.csv("airway_scaledcounts.csv", row.names = 1)
metadata <- read.csv("airway_metadata.csv")
```

Let's have a wee peak at our **counts** data.

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG00000000003	723	486	904	445	1170
ENSG00000000005	0	0	0	0	0
ENSG000000000419	467	523	616	371	582
ENSG000000000457	347	258	364	237	318
ENSG000000000460	96	81	73	66	118
ENSG000000000938	0	0	1	0	2

	SRR1039517	SRR1039520	SRR1039521
ENSG00000000003	1097	806	604
ENSG00000000005	0	0	0
ENSG000000000419	781	417	509
ENSG000000000457	447	330	324
ENSG000000000460	94	102	74
ENSG000000000938	0	0	0

and the metadata

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

Q1. How many “genes” are in this dataset?

```
nrow(counts)
```

```
[1] 38694
```

There are 38694 genes in the dataset.

Q2. How many experiments (i.e columns in `counts` or rows in `metadata`) are there?

```
ncol(counts)
```

```
[1] 8
```

There are 8 experiments.

Q3. How many “control” experiments are there in the dataset?

```
sum(metadata$dex == "control")
```

```
[1] 4
```

There are 4 control experiments in the dataset

Toy analysis example

1. Extract the “control” columns from `counts`
2. Calculate the mean value for each gene in these “control” columns 3-4. Do the same for the “treated” columns.
3. Compare these mean values for each gene.

Step 1.

```
control.inds <- metadata$dex == "control"  
control.counts <- counts[,control.inds]  
head(control.counts)
```

	SRR1039508	SRR1039512	SRR1039516	SRR1039520
ENSG000000000003	723	904	1170	806
ENSG000000000005	0	0	0	0
ENSG000000000419	467	616	582	417
ENSG000000000457	347	364	318	330
ENSG000000000460	96	73	118	102
ENSG000000000938	0	1	2	0

Step 2.

```
control.mean <- rowMeans(control.counts)
```

Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called `treated.mean`)

Step 3.

```
treated.inds <- metadata$dex == "treated"
treated.counts <- counts[, treated.inds]
head(treated.counts)
```

	SRR1039509	SRR1039513	SRR1039517	SRR1039521
ENSG000000000003	486	445	1097	604
ENSG000000000005	0	0	0	0
ENSG000000000419	523	371	781	509
ENSG000000000457	258	237	447	324
ENSG000000000460	81	66	94	74
ENSG000000000938	0	0	0	0

Step 4.

```
treated.mean <- rowMeans(treated.counts)
```

For ease of book-keeping we can store these together in one data frame called `meancounts`

Step 5.

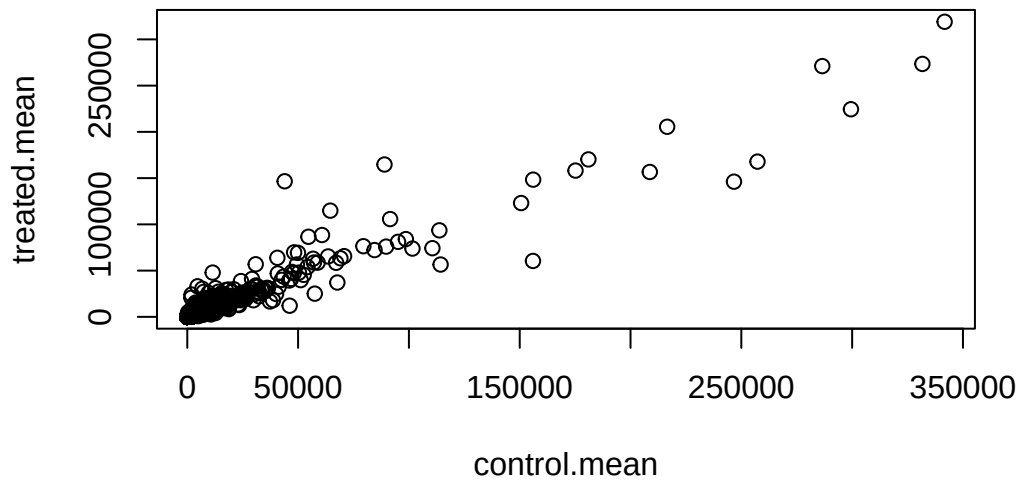
```
meancounts <- data.frame(control.mean, treated.mean)
head(meancounts)
```

	control.mean	treated.mean
ENSG000000000003	900.75	658.00
ENSG000000000005	0.00	0.00
ENSG000000000419	520.50	546.00
ENSG000000000457	339.75	316.50
ENSG000000000460	97.25	78.75
ENSG000000000938	0.75	0.00

Q5. Create a scatter plot showing the mean of the treated samples against the mean of the control samples. Your plot should look something like the following.

Plot these against each other

```
plot(meancounts)
```



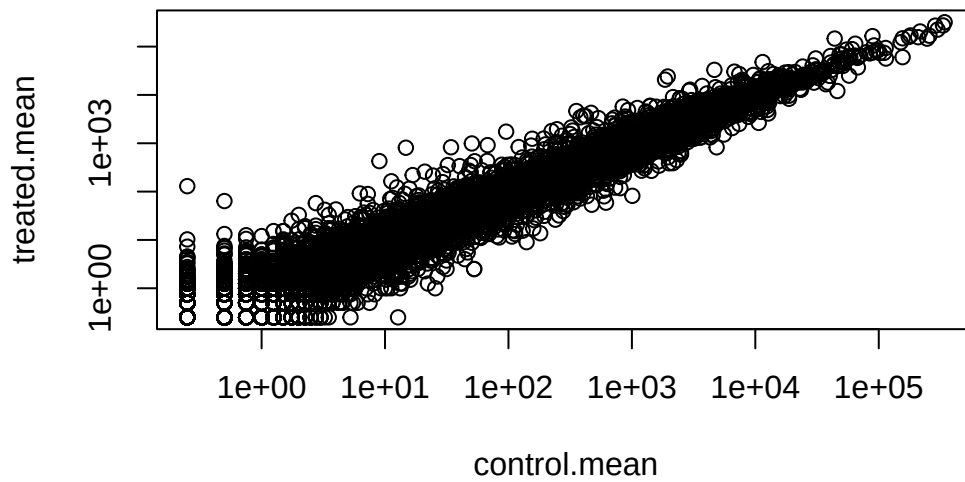
```
library(ggplot2)
```

Q6. Try plotting both axes on a log scale. What is the argument to `plot()` that allows you to do this?

```
plot(meancounts, log = "xy")
```

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15032 x values ≤ 0 omitted from logarithmic plot

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15281 y values ≤ 0 omitted from logarithmic plot



The log = “xy” argument of the plot function allows us to convert the plot into log scale
We use log2”fold-change” as a way to compare

```
# treated/control = foldchange
```

```
log2(10/10)
```

```
[1] 0
```

```
log2(20/10)
```

```
[1] 1
```

```
log2(10/20)
```

```
[1] -1
```

```
log2(40/10)
```

```
[1] 2
```

```
meancounts$log2fc <- log2(meancounts$treated.mean/meancounts$control.mean)
head(meancounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000938	0.75	0.00	-Inf

We need to remove the zero count rows from the data set before moving forward

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)

to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000971	5219.00	6687.50	0.35769358
ENSG000000001036	2327.00	1785.75	-0.38194109

Q7. What is the purpose of the arr.ind argument in the which() function call above? Why would we then take the first column of the output and need to call the unique() function?

The arr.ind argument in the which function allows the function to return both the column and row indices where there is a TRUE value. The unique function is used here to make sure a row with 0 in both columns isn't counted twice.

```
library(dplyr)
```

Warning: package 'dplyr' was built under R version 4.3.3

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
mygenes <- meancounts %>% filter((control.mean & treated.mean != 0.0))
head(mygenes)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG0000000000419	520.50	546.00	0.06900279
ENSG0000000000457	339.75	316.50	-0.10226805
ENSG0000000000460	97.25	78.75	-0.30441833
ENSG0000000000971	5219.00	6687.50	0.35769358
ENSG0000000001036	2327.00	1785.75	-0.38194109

```
y <- data.frame(a <- c(1, 5, 0, 5), b <- c(1,0,5,5))
y
```

	a....c.1..5..0..5.	b....c.1..0..5..5.
1	1	1
2	5	0
3	0	5
4	5	5

```
which(y == 0, arr.ind = T)[,1]
```

[1] 3 2

A common “rule-of-thumb” for calling something “up” regulated is a log2-fold-change of +2 or greater. For down regulated -2 or less.

Q8. How many genes are “up” regulated at the +2 log2FC threshold?

```
sum(mygenes$log2fc >= 2)
```

[1] 314

Q9. How many genes are “down” regulated at the -2 log₂FC threshold?

```
sum(mygenes$log2fc <= -2)
```

[1] 485

Q10. Do you trust these results? Why or why not?

I don't trust these results because we don't know if these genes are statistically significant. We don't know if they are “significantly” up or down regulated.

DESeq Analysis

Let's do this with DESeq2 and put some stats behind these numbers.

```
library(DESeq2)
```

Warning: package 'DESeq2' was built under R version 4.3.3

Warning: package 'GenomeInfoDb' was built under R version 4.3.3

Warning: package 'matrixStats' was built under R version 4.3.3

DESeq wants 3 things for analysis, countData, colData and design.

```
dds <- DESeqDataSetFromMatrix(countData = counts,  
                              colData = metadata,  
                              design = ~dex)
```

converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in design formula are characters, converting to factors

The main function in the DESeq package to run analysis is called DESeq().

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

Get the results out of this DESeq object with the function `results()`

```
res <- results(dds)
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

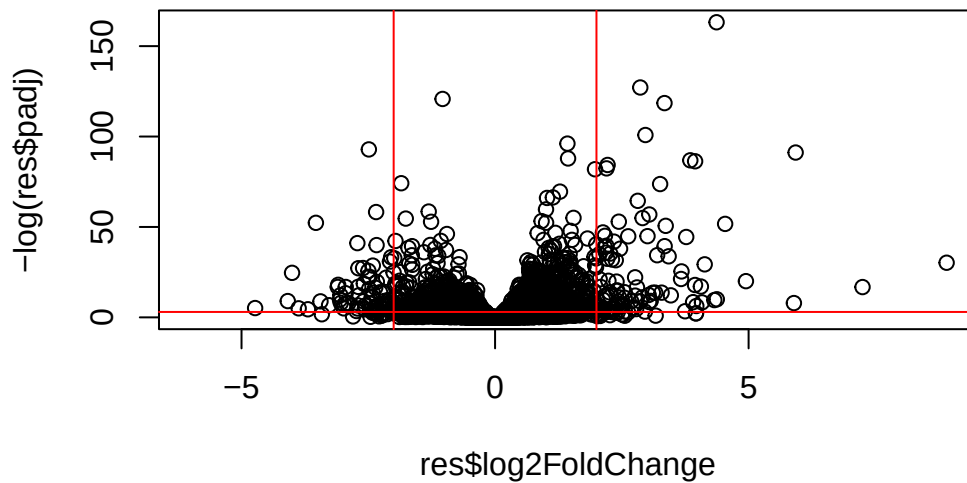
DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj				
	<numeric>				
ENSG000000000003	0.163035				
ENSG000000000005	NA				
ENSG000000000419	0.176032				
ENSG000000000457	0.961694				
ENSG000000000460	0.815849				
ENSG000000000938	NA				

Volcano Plot

This is a plot of log2FC vs adjusted p-value

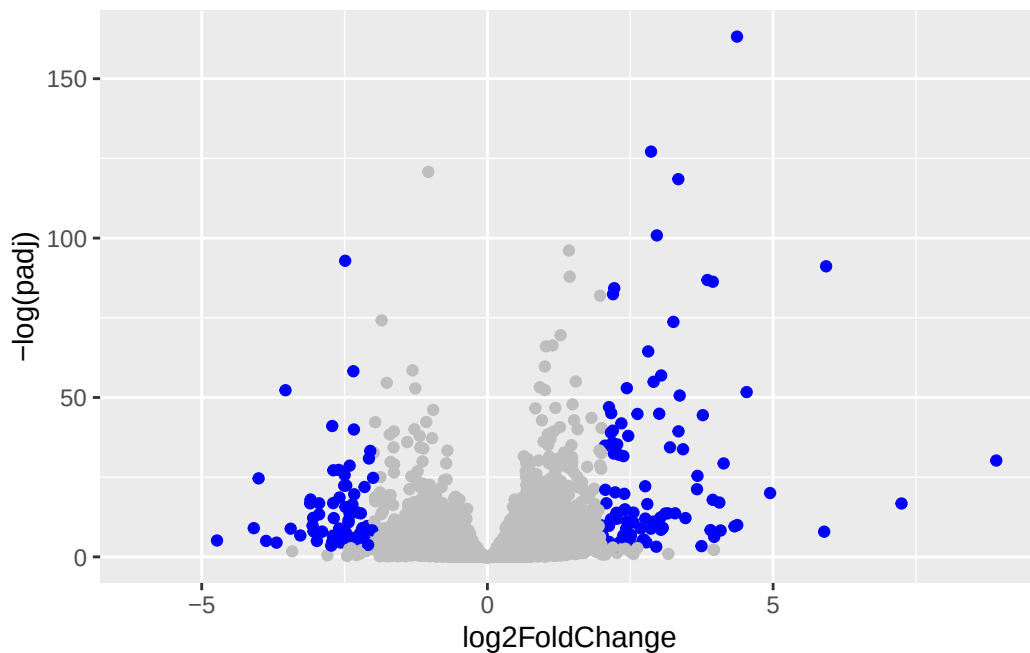
```
plot(res$log2FoldChange, -log(res$padj))
abline(v = c(-2,2), col = "red")
abline(h = -log(0.05), col = "red")
```



```
library(ggplot2)
```

```
mycols <- rep("gray", nrow(res))
mycols[abs(res$log2FoldChange) >= 2] <- "blue"
mycols[res$padj >= 0.05] <- "gray"
ggplot(res, aes(log2FoldChange, -log(padj))) +geom_point(col = mycols)
```

Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



Save our results

```
write.csv(res, file = "myresults.csv")
```

Add annotation data

We need to add gene symbols, gene names and other database ids to make my results useful for further analysis.

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106

```

ENSG000000000460 87.682625 -0.1471420 0.257007 -0.572521 0.5669691
ENSG000000000938 0.319167 -1.7322890 3.493601 -0.495846 0.6200029
      padj
      <numeric>
ENSG000000000003 0.163035
ENSG000000000005      NA
ENSG000000000419 0.176032
ENSG000000000457 0.961694
ENSG000000000460 0.815849
ENSG000000000938      NA

```

We have ENSEMBLE database ids in our `res` object.

```
head(rownames(res))
```

```

[1] "ENSG000000000003" "ENSG000000000005" "ENSG000000000419" "ENSG000000000457"
[5] "ENSG000000000460" "ENSG000000000938"

```

We can use the `mapIds()` function from `bioconductor` package.

```
library(AnnotationDbi)
```

Attaching package: 'AnnotationDbi'

The following object is masked from 'package:dplyr':

```
select
```

```
library(org.Hs.eg.db)
```

```
columns(org.Hs.eg.db)
```

```

[1] "ACCNUM"      "ALIAS"      "ENSEMBL"    "ENSEMBLPROT" "ENSEMBLTRANS"
[6] "ENTREZID"    "ENZYME"     "EVIDENCE"   "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"    "GO"         "GOALL"      "IPI"          "MAP"
[16] "OMIM"        "ONTOLOGY"   "ONTOLOGYALL" "PATH"         "PFAM"
[21] "PMID"        "PROSITE"    "REFSEQ"     "SYMBOL"       "UCSCKG"
[26] "UNIPROT"

```

```
res$symbol <- mapIds(org.Hs.eg.db,
                     keys = row.names(res), #Our genenames
                     keytype = "ENSEMBL", #The format of our genenames
                     column = "SYMBOL",    # The new format we want to add
                     multiVals = "first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res$symbol)
```

```
ENSG000000000003 ENSG000000000005 ENSG000000000419 ENSG000000000457 ENSG000000000460
      "TSPAN6"           "TNMD"           "DPM1"           "SCYL3"           "FIRRM"
ENSG000000000938
      "FGR"
```

Add GENENAME then the ENTREZ ID

```
res$genename <- mapIds(org.Hs.eg.db,
                      keys = row.names(res), #Our genenames
                      keytype = "ENSEMBL", #The format of our genenames
                      column = "GENENAME",  # The new format we want to add
                      multiVals = "first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 8 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj	symbol		genename	

	<numeric>	<character>	<character>
ENSG000000000003	0.163035	TSPAN6	tetraspanin 6
ENSG000000000005	NA	TNMD	tenomodulin
ENSG000000000419	0.176032	DPM1 dolichyl-phosphate m..	
ENSG000000000457	0.961694	SCYL3 SCY1 like pseudokina..	
ENSG000000000460	0.815849	FIRRM FIGNL1 interacting r..	
ENSG000000000938	NA	FGR FGR proto-oncogene, ..	

```
res$entrez <- mapIds(org.Hs.eg.db,
  keys = row.names(res), #Our genenames
  keytype = "ENSEMBL", #The format of our genenames
  column = "ENTREZID", # The new format we want to add
  multiVals = "first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 9 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029

	padj	symbol	genename	entrez
	<numeric>	<character>	<character>	<character>
ENSG000000000003	0.163035	TSPAN6	tetraspanin 6	7105
ENSG000000000005	NA	TNMD	tenomodulin	64102
ENSG000000000419	0.176032	DPM1 dolichyl-phosphate m..		8813
ENSG000000000457	0.961694	SCYL3 SCY1 like pseudokina..		57147
ENSG000000000460	0.815849	FIRRM FIGNL1 interacting r..		55732
ENSG000000000938	NA	FGR FGR proto-oncogene, ..		2268

Save my annotated results

```
write.csv(res, file = "myresults_annotated.csv")
```

Pathway Analysis

We will use the **gage** function from bioconductor.

```
library(pathview)
library(gage)
library(gageData)
```

What **gage** wants as input is a named vector of importance i.e. a vector with labeled fold-changes

```
data(kegg.sets.hs)

# Examine the first 2 pathways in this kegg set for humans
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
[1] "10"    "1544" "1548" "1549" "1553" "7498" "9"

$`hsa00983 Drug metabolism - other enzymes`
[1] "10"    "1066" "10720" "10941" "151531" "1548" "1549" "1551"
[9] "1553" "1576" "1577" "1806" "1807" "1890" "221223" "2990"
[17] "3251" "3614" "3615" "3704" "51733" "54490" "54575" "54576"
[25] "54577" "54578" "54579" "54600" "54657" "54658" "54659" "54963"
[33] "574537" "64816" "7083" "7084" "7172" "7363" "7364" "7365"
[41] "7366" "7367" "7371" "7372" "7378" "7498" "79799" "83549"
[49] "8824" "8833" "9"    "978"
```

```
foldchanges <- res$log2FoldChange
names(foldchanges) <- res$entrez
head(foldchanges)
```

7105	64102	8813	57147	55732	2268
-0.35070302	NA	0.20610777	0.02452695	-0.14714205	-1.73228897

```
keggres <- gage(foldchanges, gsets=kegg.sets.hs, same.dir = T)
```

What is in the results:

```
attributes(keggres)
```

```
$names
[1] "greater" "less"    "stats"
```

```
head(keggres$less, 5)
```

	p.geomean	stat.mean
hsa05332 Graft-versus-host disease	0.0004250461	-3.473346
hsa04940 Type I diabetes mellitus	0.0017820293	-3.002352
hsa05310 Asthma	0.0020045888	-3.009050
hsa04672 Intestinal immune network for IgA production	0.0060434515	-2.560547
hsa05330 Allograft rejection	0.0073678825	-2.501419
	p.val	q.val
hsa05332 Graft-versus-host disease	0.0004250461	0.09053483
hsa04940 Type I diabetes mellitus	0.0017820293	0.14232581
hsa05310 Asthma	0.0020045888	0.14232581
hsa04672 Intestinal immune network for IgA production	0.0060434515	0.31387180
hsa05330 Allograft rejection	0.0073678825	0.31387180
	set.size	exp1
hsa05332 Graft-versus-host disease	40	0.0004250461
hsa04940 Type I diabetes mellitus	42	0.0017820293
hsa05310 Asthma	29	0.0020045888
hsa04672 Intestinal immune network for IgA production	47	0.0060434515
hsa05330 Allograft rejection	36	0.0073678825

Let's look at just one of these hsa05310

```
pathview(gene.data=foldchanges, pathway.id="hsa05310")
```

Insert figure for this pathway

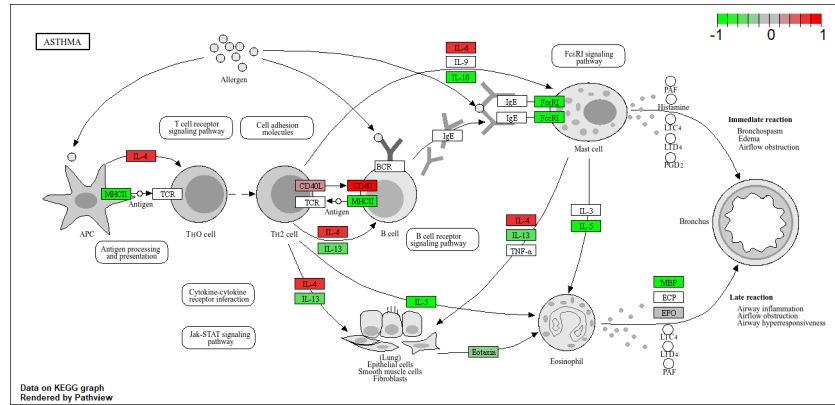


Figure 1: Asthma pathway from KEGG with my differentially expressed genes highlighted